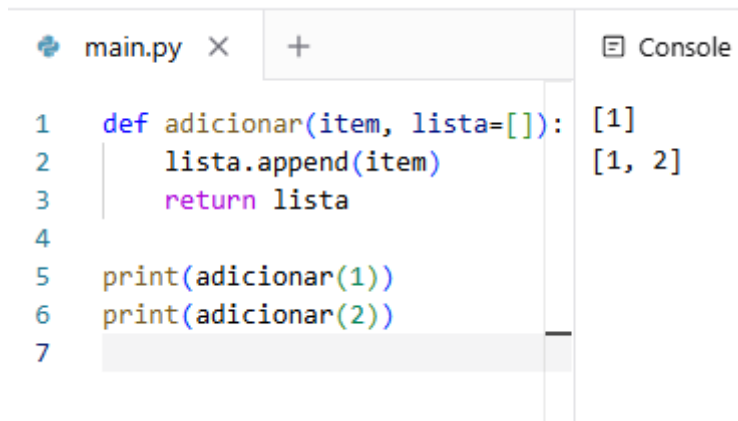


Aula 8 – Subprogramas

1. Python

Saída:



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'adicionar' that takes an 'item' and a default list 'lista=[]'. The function appends the item to the list and returns it. Below the function definition, there are two print statements: 'print(adicionar(1))' and 'print(adicionar(2))'. To the right of the code editor is a 'Console' window showing the output of the program. The first line of output is '[1]' and the second line is '[1, 2]'. The console output is aligned with the corresponding print statements in the code.

```
1 def adicionar(item, lista=[]):  
2     lista.append(item)  
3     return lista  
4  
5 print(adicionar(1))  
6 print(adicionar(2))  
7
```

Console

[1]
[1, 2]

Conceito: parâmetro padrão mutável.

A lista padrão é criada uma única vez e continua sendo usada nas próximas chamadas. Por isso, o 2 é adicionado à mesma lista.

2. Java

Saída:

```
OneCompiler  
Main.java  
1 public class Main {  
2  
3     static void zera(int[] v, int n) {  
4         v[0] = 0;  
5         n = 0;  
6     }  
7  
8     public static void main(String[] args) {  
9         int[] v = {5, 5};  
10        int n = 5;  
11  
12        zera(v, n);  
13  
14        System.out.println(v[0] + " " + n);  
15    }  
16 }
```

Console
I/O
0 5

Conceito: passagem de parâmetros por valor.

O vetor é um objeto, então a alteração em `v[0]` afeta o vetor original. Já `n` é um valor primitivo e sua cópia é alterada apenas dentro da função.

3. Python

Saída:

```
OneCompiler  
main.py  
1 fs = [lambda: i for i in range(3)]  
2 print([f() for f in fs])  
3
```

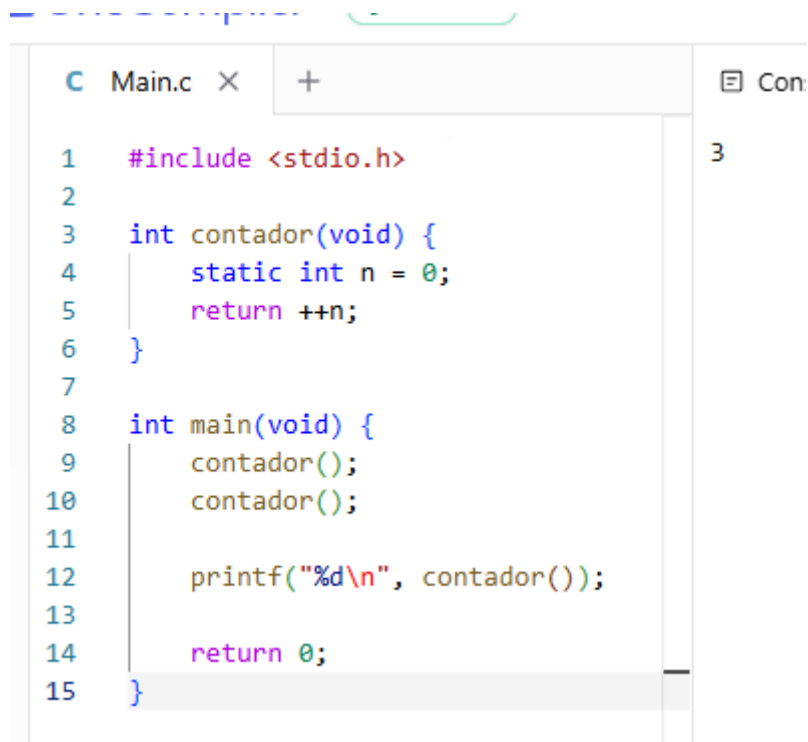
Console
[2, 2, 2]

Conceito: fechamento (closure).

As funções lambda capturam a variável `i`, e não uma cópia do valor em cada repetição. Quando são executadas, o `for` já terminou e `i` vale 2.

4. C

Saída:



```
1  #include <stdio.h>
2
3  int contador(void) {
4      static int n = 0;
5      return ++n;
6  }
7
8  int main(void) {
9      contador();
10     contador();
11
12     printf("%d\n", contador());
13
14     return 0;
15 }
```

The screenshot shows a code editor window titled 'Main.c'. The code defines a function `contador` that uses a static variable `n` to maintain its state across calls. In `main`, `contador` is called twice, and then its value is printed. The output '3' is visible on the right side of the editor.

Conceito: variável estática.

A variável `n` mantém seu valor entre as chamadas. As duas primeiras chamadas retornam 1 e 2, então a terceira retorna 3.

5. Rust

Saída:

```
1 fn dobra(v: Vec<i32>) -> Vec<i32> {
2     v.iter().map(|x| x * 2).collect()
3 }
4 let v = vec![1, 2, 3];
5 let d = dobra(v);
6 println!("{:?} {:?}", v, d);
7
```

error: expected item, found keyword `let`
--> HelloWorld.rs:4:1
|
4 | let v = vec![1, 2, 3];
| ^^^
|
|
| `let` cannot be used for global variables
| help: consider using `static` or `const` instead.
|
= note: for a full list of items that can appear

error: aborting due to 1 previous error

Debug this error

Conceito: passagem por posse (move).

Ao chamar `dobra(v)`, o vetor `v` é movido para a função. Por isso, não é possível usar `v` novamente no `println`. O programa não chega a imprimir os vetores.

Se feito corretamente, retorna `[1, 2, 3] [2, 4, 6]`

6. Python

Saída:

```
total = 0

def adiciona(x):
    total = total + x
    return total

print(adiciona(5))
```

Traceback (most recent call last):
File "/box/main.py", line 7, in <module>
 print(adiciona(5))
 ^^^^^^^^^
File "/box/main.py", line 4, in adiciona
 total = total + x
 ^^^^^
UnboundLocalError: cannot access local variable 'total'

Debug this error

Conceito: variável local e escopo.

Como existe uma atribuição a `total` dentro da função, o Python considera `total` uma variável local. Assim, `total = total + x` tenta usar a variável local antes de ela receber um valor.

Se feito certo, retorna 5.